DEL 1: PROSJEKTRESULTAT

Smidig Prosjekt

DECEMBER 19, 2025

GRUPPE 08
Patrick Antonio Vidrequin, Sindre Vethe, Even Garli, Krister Eriksen, Thanh Dat Doan, Nora Jellum, Ingrid Skoge, Ingrid Nienhuis

Contents

Egenerklæring om KI-hjelpemidler 3
1 Beskrivelse av Løsningen: 4
Main (Main.html) 4
Benchmark (Benchmark.html) 4
Underleverandør (Send-skjema.html) 4
Underleverandør form (supplier-form.html) 4
Prosjekt (project.html) 4
Profil (profile.html) 4
Nytt prosjekt (newproject.html) 5
Registrer (signup.html) 5
Login (login.html) 5
Login og Signup 6
Hovedside Beskrivelse 7
Profilside Beskrivelse 9
AI-Rapport 11
Funksjonell beskrivelse 11
Visuell Beskrivelse 11
Teknisk beskrivelse 12
Underleverandør-modul 14
Funksjonell beskrivelse 14
Visuell beskrivelse 14
Teknisk beskrivelse 16
Miljørapport 17
1. Beskrivelse av løsningen 17
2. Kode-dokumentasjon 19
Benchmarking 21
Funksjonell beskrivelse: 21
Visuell Beskrivelse: 21
Teknisk Beskrivelse: 21
Sikkerhetsanalyse sikkerhet vurdering og implementasjon 23
Innledning 23
Overordnet sikkerhetsdesign 23
Autentisering og håndtering av passord 23
Beskyttelse av sensitiv informasjon 24
Klient–server-sikkerhet 25

Tilgangskontroll og autorisasjon 25
Vurdering av svakheter og forbedringspotensial 25
Konklusjon 25
2 Leveransetest 27
Formål og metode 27
Gjennomføring 27
Resultat 27
3 Egen vurdering av resultat 28
Hva gruppen er fornøyd med 28
Samarbeid og teamdynamikk 28
Teknisk utvikling 28
Prosess og planlegging 28
Kommunikasjon 28
Hva som kunne vært bedre 28
Tidsplanlegging og estimering 28
Tekniske utfordringer 28
Kommunikasjon og koordinering 28
Scrumwise-bruk 28
Forbedringsområder og læring 29
Planlegging 29
Verktøybruk 29
Kommunikasjon 29
Scrumwise 29
Innsikt og forståelse 29
4 Vedlegg 30
Kildekode 30
Leveransetest 30
Artefakter 30
Referanser: 31

Egenerklæring om KI-hjelpemidler

Last dette dokumentet opp som vedlegg til besvarelsen din for å beskrive eventuell bruk av KI-baserte hjelpemidler i utarbeidelsen av besvarelsen.

Hensikten med skjemaet er å redegjøre for bruken av KI i oppgaven. Dette for å bevisstgjøre deg som student om din egen bruk, og for at sensor skal kunne vurdere bruken fra et metodeperspektiv. Les mer om bruk av KI i oppgaveskriving.

Bruk av verktøy som generativ kunstig intelligens (KI) kan ansees som fusk i henhold til Kristianias forskrift. Les om fusk på eksamen i Studentservice

Sjekk om det er særskilte krav for bruk eller dokumentasjon av KI-bruk i den aktuelle eksamensoppgaven.

1. Bruk av KI-baserte hjelpemidler Har det i utarbeidingen av denne besvarelsen blitt anvendt KI-baserte hjelpemidler?

☐ Ja

Chatgpt versjon 5.2 ble brukt som støtte til å formulere og forbedre tekst, samt til å få veiledning og tips knyttet til dokumentstruktur.

2. Tekstgenerering - Er deler av teksten moderert eller oversatt av ett eller flere KI-verktøy?

☐ Ja

ChatGPT ble brukt til å foreslå formuleringer og gi språklig hjelp, men innhold og faglige vurderinger er gjort av gruppen medlemmene.

3. Litteraturgjennomgang - Er deler av litteraturgjennomgangen basert på ett eller flere KI-verktøy?

☐ Nei

4. Dataanalyse - Er hele eller deler av analysen gjennomført med hjelp av KI-verktøy? Er tabeller eller lignende fremstilt av KI?

☐ Nei

5. Bildegenerering - Er ett eller flere av illustrasjonene i besvarelsen blitt generert av KI verktøy?

☐ Nei

6. Koding - Er deler av eventuell kode generert av KI verktøy?

☐ Ja

ChatGPT ble brukt til å foreslå eller hjelpe med feilsøking av kode

7. Andre KI-verktøy - Har andre typer av verktøy blitt anvendt?

☐ Nei

Ved opplasting og innlevering av egenerklæringen bekrefter jeg at jeg er kjent med Kristianias regler for bruk av kunstig intelligens og at jeg har redegjort for all bruk av kunstig intelligens.

1 Beskrivelse av Løsningen:

Main (Main.html)

- /api/projects (getProjects, getAllSubmissions)

Benchmark (Benchmark.html)

- /api/projects (getProjects, getAllSubmissions)

Underleverandør (Send-skjema.html)

- /api/projects (getProjects, getAllSubmissions)
- /api/send-invite (createSubmissionInvite)
- /api/submissions (getAllsubmissions)

Underleverandør form (supplier-form.html)

- /api/form/XXXX (getSubmissionsByToken)
- /api/submit (getSubmissionsByToken, completeSubmissions)

Prosjekt (project.html)

- /project-data?id=XXXX (getProjects, getAllSubmissions)
- /upload-file (updateProjectFile)
- /generate-report (getProject, saveProjectReport)
- /api/supplier-report/projectID/supplierID (getSupplierForReport)

Profil (profile.html)

- /getProfile (getProfileInfo)
- /getUnderleverandor (getUnderleverandør)

Nytt prosjekt (newproject.html)

- /createProject (createProject)

Registrer (signup.html)

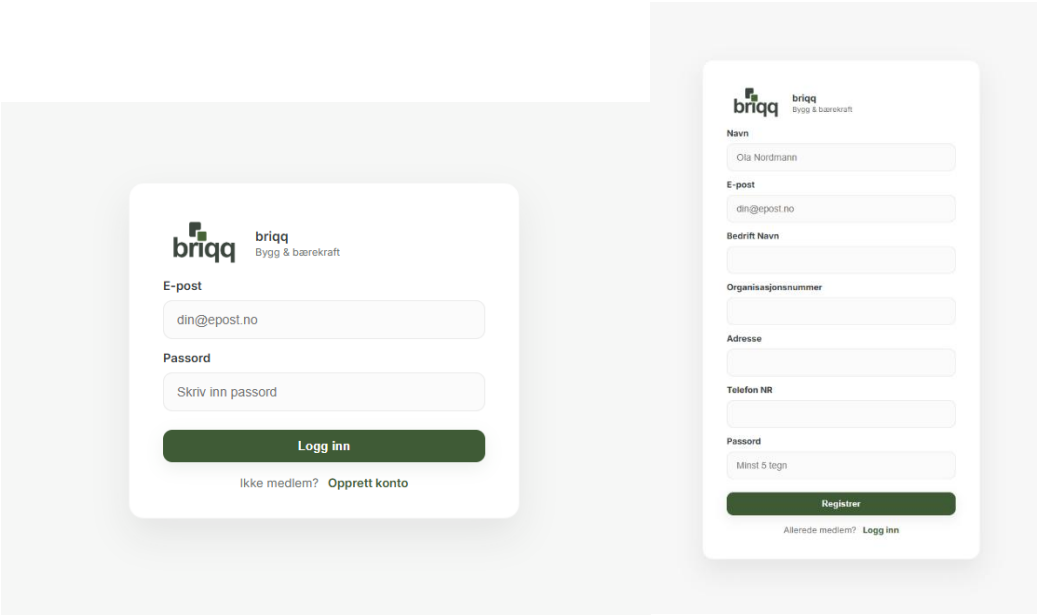
- /signUp (signUp)

Login (login.html)

- login (login)

Login og Signup

Steg	Beskrivelse	Teknisk Beskrivelse
1. Åpne nettsiden	Bruker går inn på siden der man kan lage en ny bruker eller innlogging siden.	Routes på backend henter fram signup.html eller login.html og sender til frontend.
2. Registrer bruker	Bruker fyller ut skjemaet med nødvendig informasjon for å registrere konto og trykker på "Registrer" knappen.	Skjemaet er koblet til "/signup". Ved registrering sendes skjemaet til backend. På backend brukes "name" verdien på skjemaet til å velge felt fra skjemaet. f.eks "req.body.epost". Funksjonen signUp i "db.js" filen setter inn alle feltene i databasen. Ved en feil sendes bruker til "/signup.html?error=1" for å vise en feilmelding på frontend.
3. Logg inn	Bruker fyller inn skjemaet med e-post og passord.	Skjemaet er koblet til "/login" Ved innlogging sendes skjemaet til backend der name attributtet brukes for å velge ut spesifikke felt. "login" funksjonen i "db.js" finner en bruker i databasen basert på epost og passord. Vist serveren ikke finner brukeren blir bruker sendt til "/?error=login_failed" eller til "/?error=server_error" ved en server feil.
4. Vidresending	Bruker videresendes til hovedsiden.	Ved en suksessfull login sender serveren brukeren videre til hovedsiden (server side rendering) og setter emailen til brukeren i cookiesen for videre bruk.

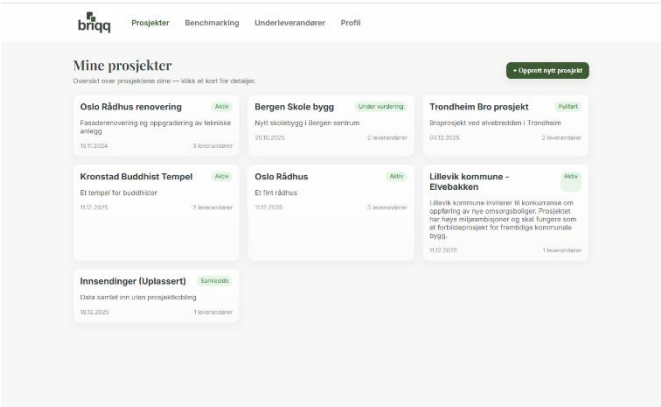


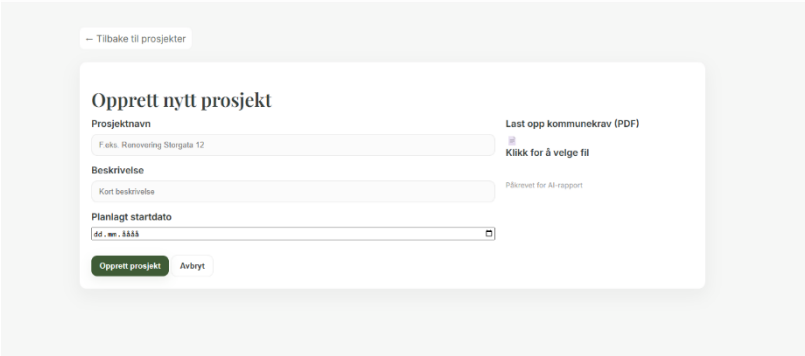
Siden for å logge inn og siden for å registrere nye kontoer har et relativt likt design. Siden for å registrere en bruker har flere felter enn login siden som er hovedforskjellen. Begge sidene bruker samme css-fil for å designe layouten og formen. Sidene har et div element som fungerer som en konteiner som inneholder en HTML form med forskjellige felter, logoen og en egen div for å vise fram error messages.

Hovedside Beskrivelse

Steg	Beskrivelse	Teknisk Beskrivelse
1. Åpne hovedsiden	En bruker åpner eller blir videresendt til hovedsiden etter en suksessfull innlogging	Siden lastes inn og "/api/projects" API endepunktet blir callet. Backend bruker emailen lagret i cookisen og bruker "getProjects" funksjonen i "db.js" for å hente fram alle prosjekter tilhørende brukeren. Backend sender alle prosjektene i JSON format til frontend. Prosjektene loopes igjennom og oppretter en HTML "article" og fyller inn informasjonen tilhørende prosjektet.

2. Registrerer et nytt prosjekt	En bruker oppretter et nytt prosjekt	HTML formen er koblet opp mot "/createProject" API endepunkte. Når bruker trykker på "opprett prosjekt" knappen så sendes all informasjonen i HTML formen til backend. Backend bruker "createProject" funksjonen i "db.js" filen og sender alt inn i databasen
3. Åpne prosjekt	Bruker åpner trykker inn på et eksisterende prosjekt på hovedsiden.	Alle prosjektene på hovedsiden har en HTML id attribut. Når en bruker trykker på et prosjekt så sendes brukeren videre til "/project?id=XXXX" der IDen i URLen er IDen til prosjektet.

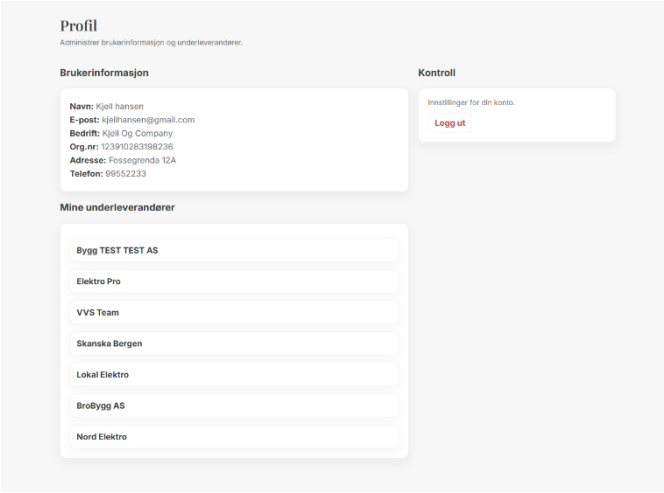




Opprett nytt prosjekt siden er også en div som inneholder en form. Feltet i formen som brukes for å velge dato har "type=date" som gir en grafisk kalender brukeren kan bruke for å velge dato. Hovedsiden består av en section som laster inn alle HTML-artikler ved Javascript i en grid til de forskjellige prosjektene.

Profilside Beskrivelse

Steg	Beskrivelse	Teknisk Beskrivelse
1. Bruker åpner profilsiden	Bruker åpner opp profilsiden å underleverandør og profilinformasjon lastes inn	To funksjoner kjører når bruker åpner profilsiden. “loadprofile” og “loadsuppliers”. Loadprofile funksjonen fetcher /getProfile API endepunktet. Backend bruker “getProfileInfo” funksjonen i “db.js” filen og henter fram all informasjon tilhørende bruker og sender til frontend i JSON format. Frontend bruker javascript til å sette profilinformasjonen til de forskjellige feltene. “loadsuppliers” fetcher “/getUnderleverandor” API endepunktet. Backend bruker “getUnderleverandør” funksjonen i “db.js” filen og henter fram alle underleverandører tilhørende bruker og sender det til frontend i JSON format. Frontend looper igjennom dataen og laster alt inn i en egen div for undeleverandører.
2. Bruker logger ut	Bruker trykker på “logg ut” knappen og blir logget ut av nettsiden.	“Logg ut” knappen er linket til “/logout”. Når bruker trykker på logg ut blir brukeren sendt videre til “logout”. routen “/logout” på backend sletter email lagret i cookies og sender brukeren videre til hoved routen. Der sjekkes det vist det er lagret en email i cookies og vist ikke blir brukeren sendt til innloggingssiden.



Profilsiden består av en div som inneholder alt som tilhører profilsiden. Inni denne diven er det tre “section” elementer. Den ene section elementet brukes for å vise profil informasjon. Den andre brukes for å vise innstillinger for kontoen. Foreløpig består det bare av muligheten til å logge ut av kontoen. Det siste section elementet inneholder en tom div som fylles opp med flere div elementer som inneholder en underleverandør. Underleverandører lastes inn ved hjelp av javascript.

AI-Rapport

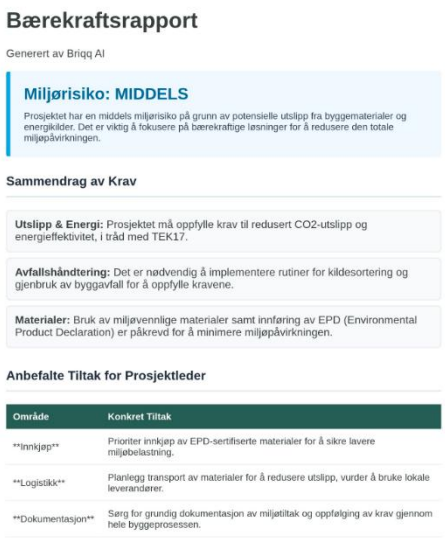
Funksjonell beskrivelse

Denne løsningen er en web-applikasjon som hjelper med miljøvurderinger og rapportering. Brukeren lager et prosjekt og laster opp et kravdokument i PDF format. AI-modellen vil da generere en strukturert miljørisikorapport basert på kravdokumentet som ble lastet opp.

AI blir brukt til å:

- Estimere CO2 utslipp basert på materialer.
- Analysere kravdokument og finne sentrale miljøkrav innen utslipp, energi, avfall og materialvalg.
- Generere anbefalte tiltak, som blir presentert på en forståelig måte.

Resultatet blir lagret som en ferdig rapport, og er lagret i selve prosjektet.



Bærekraftsrapport

Generert av Briqq AI

Miljørisiko: MIDDELS

Prosjektet har en middels miljørisiko på grunn av potensielle utslipp fra byggematerialer og energikilder. Det er viktig å fokusere på bærekraftige løsninger for å redusere den totale miljøpåvirkningen.

Sammendrag av Krav

Utslipp & Energi: Prosjektet må oppfylle krav til redusert CO2-utslipp og energieffektivitet, i tråd med TEK17.

Avfallshåndtering: Det er nødvendig å implementere rutiner for kildesortering og gjenbruk av byggavfall for å oppfylle kravene.

Materialer: Bruk av miljøvennlige materialer samt innføring av EPD (Environmental Product Declaration) er påkrevd for å minimere miljøpåvirkningen.

Anbefalte Tiltak for Prosjektleder

Område	Konkret Tiltak
Innkjøp	Prioriter innkjøp av EPD-sertifiserte materialer for å sikre lavere miljøbelastning.
Logistikk	Planlegg transport av materialer for å redusere utslipp, vurder å bruke lokale leverandører.
Dokumentasjon	Sørg for grundig dokumentasjon av miljøtiltak og oppfølging av krav gjennom hele byggeprosessen.

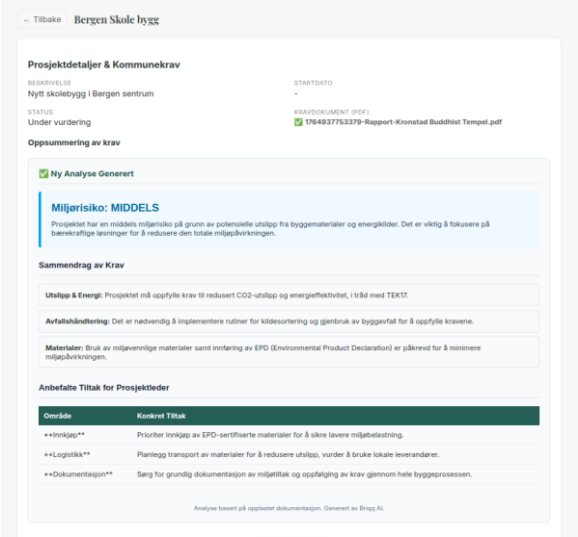
Visuell Beskrivelse

Panelet er bygget med enkel og oversiktlig design i tankene, der tydelig seksjonering og visuelle virkemidler støtter lesbarheten av panelet.

Rapporten presenteres med:

- Tydelige overskrifter og inndeling av seksjoner
- Fargekoder som speiler fokuset på grønne løsninger.

- Kortfattede sammendrag i bokser og tabeller for tiltak.


← Tilbake Bergen Skole bygg

Prosjektdetaljer & Kommunekrav

BESKRIVELSE
Nytt skolebygg i Bergen sentrum

STARTDATO
-

STATUS
Under vurdering

KRAVDOKUMENT (PDF)
✅ 1764937753379-Rapport-Kronstad Buddhist Tempel.pdf

Oppsummering av krav

✅ **Ny Analyse Generert**

Miljørisiko: MIDDELS

Prosjektet har en middels miljørisiko på grunn av potensielle utslipp fra byggematerialer og energikilder. Det er viktig å fokusere på bærekraftige løsninger for å redusere den totale miljøpåvirkningen.

Sammendrag av Krav

Utslipp & Energi: Prosjektet må oppfylle krav til redusert CO2-utslipp og energieffektivitet, i tråd med TEK17.

Avfallshåndtering: Det er nødvendig å implementere rutiner for kildesortering og gjenbruk av byggavfall for å oppfylle kravene.

Materialer: Bruk av miljøvennlige materialer samt innføring av EPD (Environmental Product Declaration) er påkrevd for å minimere miljøpåvirkningen.

Anbefalte Tiltak for Prosjektleder

Område	Konkret Tiltak
Innkjøp	Prioriter innkjøp av EPD-sertifiserte materialer for å sikre lavere miljøbelastning.
Logistikk	Planlegg transport av materialer for å redusere utslipp, vurder å bruke lokale leverandører.
Dokumentasjon	Sørg for grundig dokumentasjon av miljøtiltak og oppfølging av krav gjennom hele byggeprosessen.

Analyse basert på opplastet dokumentasjon. Generert av Briqq AI.

Rapporten bruker HTML og CSS styling for å fremstå mer profesjonell, og hjelper rapporten til å være konsistent og enkelt leselig.

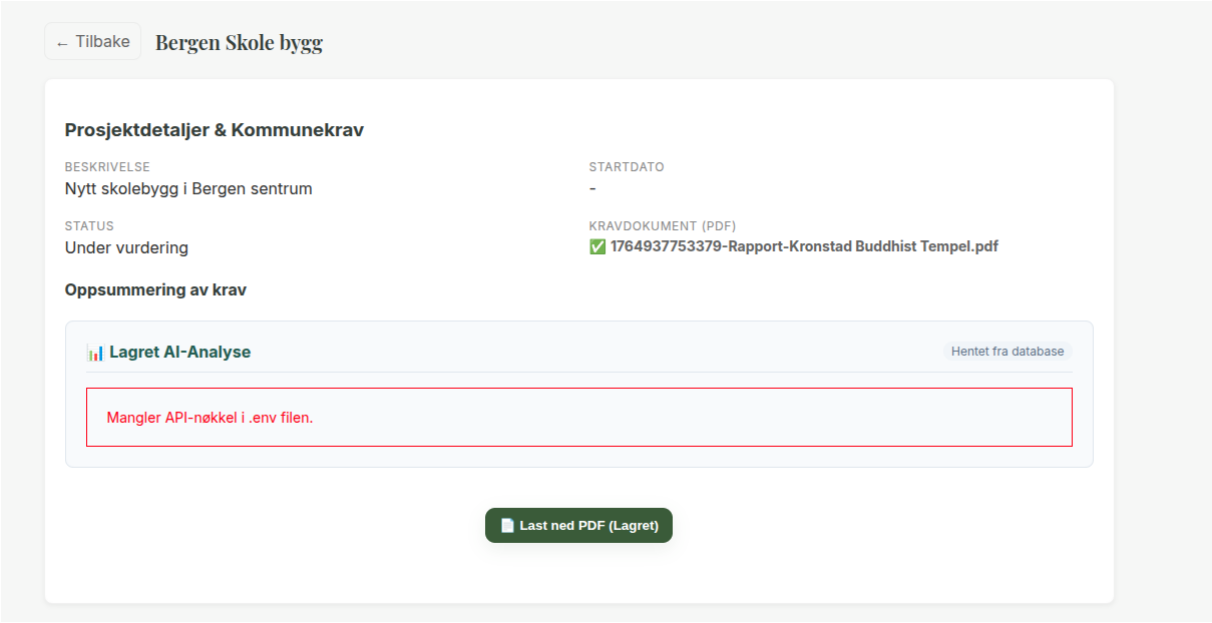
Teknisk beskrivelse

Løsningen bruker JavaScript, som kommuniserer med en API fra OpenRouter for tilgang på språkmodeller, som er ansvarlig for analyse og rapportgenerering. API-nøkkelen er sikkert integrert ved bruk av .env fil.

Arkitekturen inkluderer:

- En API som kommuniserer med openrouter for generering av innhold.
- Funksjonalitet for estimering av CO2-faktorer.
- Prompt engineering for å faktisk få innholdet man er ute etter.
- Sikker håndtering av API-er.
- Lagring av PDF-filer.

Dersom API-nøkkel mangler, vil ikke rapporten inneholde noe generert innhold, men bare skjelettet til rapporten.



Underleverandør-modul

Underleverandør-modulen har som hovedmål å effektivisere innsamlingen av miljødata fra eksterne samarbeidspartnere, og å gjøre denne prosessen både sikker og sporbar. Tradisjonelt samles slike data manuelt via Excel-skjemaer og e-post, noe som kan føre til feil og forsinkelser. Vår løsning digitaliserer hele arbeidsflyten, slik at administrator kan sende invitasjoner, underleverandør fyller ut et digitalt skjema, og innsendte data lagres direkte i prosjektets database.

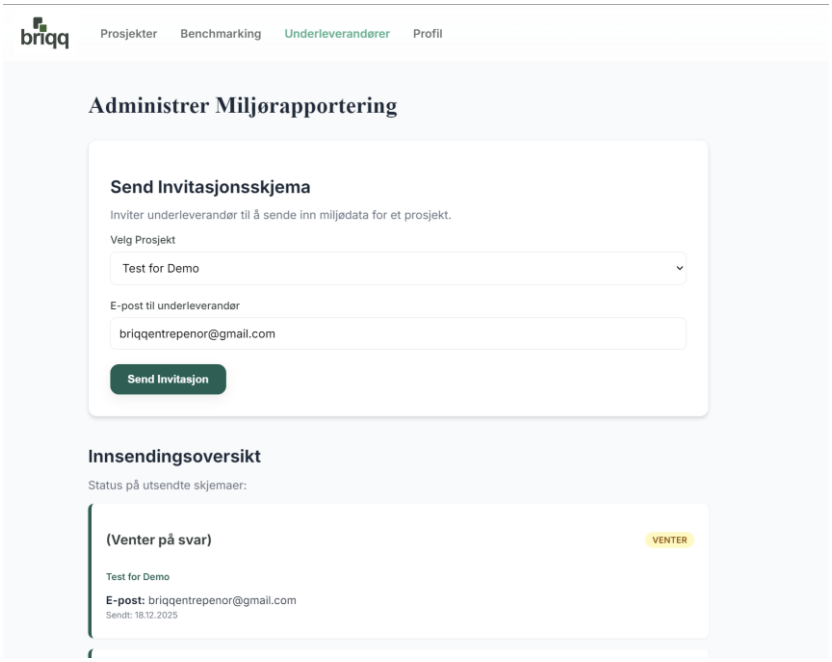
Funksjonell beskrivelse

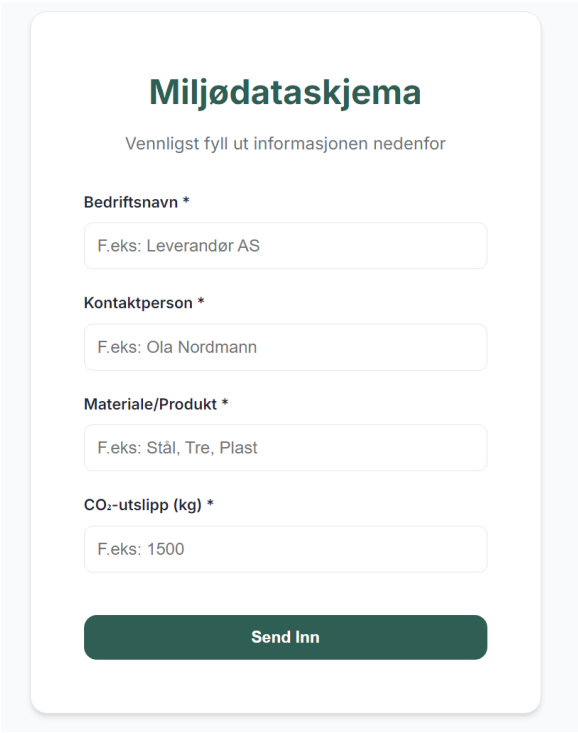
- **Invitasjon og token-generering:** Administrator velger et prosjekt i dashbordet og oppgir e-postadressen til underleverandøren. Systemet genererer et unikt UUID-token som kobles til prosjektet og e-postadressen i databasen.
- **Sikker tilgang:** Underleverandøren mottar en e-post med en direkte lenke til skjemaet (supplier-form.html token=UUID). Når lenken åpnes, validerer serveren tokenet via API-endepunktet GET /api/form/:token for å sikre at skjemaet er gyldig og ikke allerede innsendt. Ingen brukerkonto er nødvendig.
- **Datainnsamling:** Underleverandøren fyller ut bedriftsnavn, kontaktperson, materiale/produkt og CO2-utslipp i skjemaet.
- **Innsending og låsing av data:** Når skjemaet sendes inn (POST /api/submit), oppdateres databasen med dataene og status endres til "completed". Tokenet kan ikke brukes på nytt, noe som sikrer dataintegritet. Administrator kan deretter se alle innsendte data gjennom dashbordet (/send-skjema.html) med oversikt over status og rapporterte CO2-utslipp.

Visuell beskrivelse

- **Administrator-grensesnittet:** Dashbordet viser prosjekter og innsendinger i et oversiktlig format. Status for hvert skjema markeres med fargede badges (“Venter” eller “Mottatt”), og grunnleggende informasjon som e-post og CO2 vises. Det er enkelt å velge prosjekt og sende invitasjoner.
- **Underleverandør-skjema:** Designet er minimalistisk og fokusert på skjemaet. Siden har ingen navigasjonsmeny, kun nødvendige inputfelt med tydelige etiketter og placeholder-tekst.
    - Skjemaet er responsivt, og fungerer både på mobil og desktop.

- Det gir interaktiv tilbakemelding: feilaktig eller mangelfull utfylling markeres tydelig, og ved vellykket innsending vises en bekreftelsesskjerm med grønn hake og melding om at dataen er mottatt.
- Fargevalg og kontraster er nøye valgt for å sikre universell utforming.





Teknisk beskrivelse

- **Backend:** Modulen er implementert i Node.js med Express, og data lagres i MongoDB.
    - POST /api/send-invite: Genererer UUID-token, lagrer invitasjon i SubmissionsCollection, og sender e-post via Nodemailer.
    - GET /api/form/:token: Validerer token og returnerer skjemainformasjon til frontend.
    - POST /api/submit: Mottar innsendt data, oppdaterer status i SubmissionsCollection til "completed" og lagrer dataene i databasen.
    - GET /api/submissions: Henter alle innsendinger for en gitt administrator og viser status, e-post og CO2.
- **Frontend:**
    - **Dashbord** (send-skjema.html) lar administrator velge prosjekt, sende invitasjoner og se status på innsendelser.
    - **Underleverandør-skjema** (supplier-form.html) viser kun nødvendige felter, validerer input lokalt, og kommuniserer med backend via API.
- **Sikkerhet og dataintegritet:**
    - UUID-token er kryptografisk sikker og uforutsigbar.
    - Innsendte skjemaer kan ikke brukes på nytt.
    - API-endepunkter krever at administrator er logget inn, og alle handlinger logges med tidsstempel.

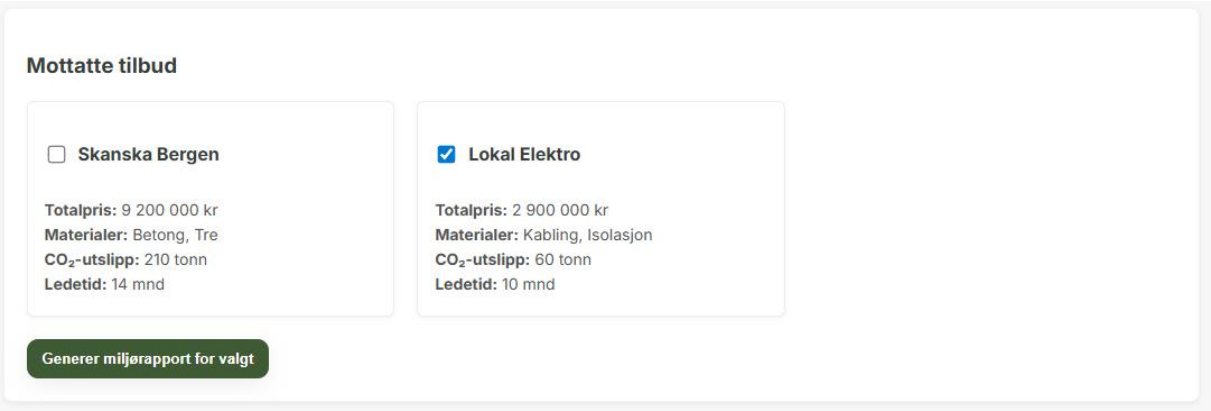
Miljørapport

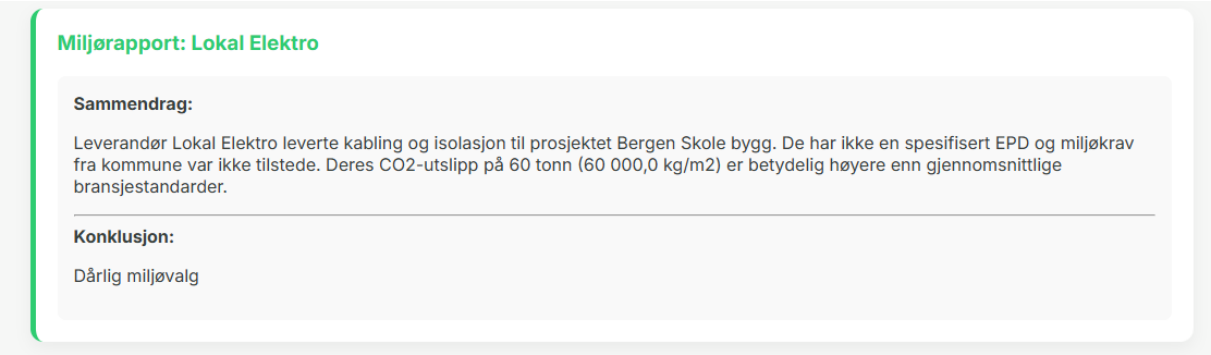
1. Beskrivelse av løsningen

Funksjonell løsning

Løsningen er en spesialisert AI-modul utviklet for å automatisere miljømessig evaluering av tilbud fra underleverandører i prosjekter hos Briqq. Hovedformålet er å transformere tekniske data som CO2-utslipp, materialvalg og EPD-status til en lettforståelig oversikt for brukerne.

De tre hovedfunksjonene i løsningen består av en automatisert utslippsanalyse, en digital kravverifisering og en strategisk anbefaling for miljøvalg. Gjennom den automatiserte utslippsanalysen beregner systemet miljøbelastning per kvadratmeter ved å sammenligne leverandørens utslippsdata direkte med prosjektets areal. Videre utfører AI-en en nøyaktig kravverifisering der leverandørens profil sammenlignes mot spesifikke miljøkrav som er ekstrahert fra kommunale PDF-dokumenter. Til slutt genererer modulen en anbefaling i form av en konkret konklusjon kategorisert som god, middels eller dårlig miljøvalg.





Visuell løsning

Brukergrensesnittet er integrert i prosjektoversikten (project.html) for å gi en sømløs arbeidsflyt.

- **Mottatte tilbud:** Underleverandører vises som oversiktlige kort som inneholder nøkkeldata som totalpris, materialer, CO_2-utslipp og ledetid.
- **Rapportvisning:** Resultatet fra AI-analysen presenteres i en strukturert rapport som kan vises direkte på skjermen eller eksporteres som en PDF-fil for dokumentasjon i anbudsprosesser.
- **Interaktivitet:** Brukeren kan velge spesifikke underleverandører fra en nedtrekksmeny og generere rapporter basert på de valgte tilbudene.

Teknisk løsning

Teknisk er løsningen bygget som en asynkron tjeneste i Node.js som kommuniserer med avanserte språkmodeller.

- **AI-Motor:** Modulen benytter DeepSeek-R1-Distill-Llama-70B via OpenRouter API-et. Denne modellen er valgt for sin evne til logisk resonnering og nøyaktig JSON-generering.
- **Robust databehandling:** Siden AI-modeller kan returnere uønsket tekst (som "tankeprosesser" i <think>-tags), inneholder koden logikk for å rense og validere svaret før det parses til JSON.
- **Arkitektur:** Tjenesten mottar et sammensatt objekt med prosjektdata (fra MongoDB) og leverandørdata, som deretter prosesseres gjennom en strukturert "prompt" for å sikre konsistente analyser.

2. Kode-dokumentasjon

Screenshot 1:

```
8   export async function generateSupplierReport(data) {
9     const { project, supplier } = data;
10
11    const supplierCO2PerM2 = (supplier.co2 * 1000) / (project.area || 1);
12
13    const prompt = `Du er en MILJØKONSULENT. Lag en JSON-rapport om denne underleverandøren.
14
15  VIKTIG: Svar KUN med gyldig JSON. Ingen <think>-tags, ingen markdown, ingen intro-tekst.
16
17  DATA:
18  - Prosjekt: ${project.title} (${project.area} m2)
19  - Leverandør: ${supplier.name}
20  - Kontakt: ${supplier.contactPerson || supplier.navn}
21  - CO2: ${supplier.co2} tonn (${supplierCO2PerM2.toFixed(1)} kg/m2)
22  - EPD: ${supplier.epd ? 'Ja' : 'Nei'}
23  - Materialer: ${supplier.materials}
24  - Miljøkrav fra kommune: ${project.aiAnalysis?.krav?.join(', ') || 'Ingen spesifisert'}
25
26  OPPGAVE:
27  Fyll ut denne JSON-strukturen basert på dataene over:
28  {
29    "sammendrag": "Kort beskrivelse av leverandøren og deres miljøprofil.",
30    "co2_analyse": "Vurdering av utslippsnivået opp mot bransjestandard.",
31    "baerekraft_dokumentasjon": "Vurdering av EPD og sertifiseringer.",
32    "konklusjon": "En avsluttende anbefaling (God/Middels/Dårlig miljøvalg)."
33  }`;
```

Forklaring: Dette bildet viser hvordan jeg har definert rollen til AI-en som en "MILJØKONSULENT". Koden viser instruksene som tvinger modellen til å svare i et strengt JSON-format. Dette er kritisk for at systemet skal kunne behandle informasjonen automatisk uten manuelle feil. Her ser man også hvordan variabler som CO2 utslipp per kvadratmeter og EPD-status blir matet inn i analysen.

Screenshot 2:

```
60      let text = response.data.choices[0].message.content;
61
62
63      text = text.replace(/<think>[\s\S]*?<\/think>/g, "");
64      text = text.replace(/```json/g, "").replace(/```/g, "");
65      text = text.trim();
66
67      const start = text.indexOf('{');
68      const end = text.lastIndexOf('}');
69
70      if (start === -1 || end === -1 || end <= start) {
71        throw new Error('AI returnerte ikke gyldig JSON format');
72      }
73
74      text = text.substring(start, end + 1);
75
76      const aiReport = JSON.parse(text);
77      console.log('✅ Leverandør-rapport generert!');
78      return aiReport;
79
80    } catch (error) {
81      console.error('❌ AI Error:', error.message);
82      return {
83        sammendrag: `Kunne ikke generere rapport for ${supplier.name}.`,
84        co2_analyse: `Data: ${supplier.co2} tonn CO2.`,
85        baerekraft_dokumentasjon: `EPD status: ${supplier.epd ? 'Ja' : 'Nei'}.`,
86        konklusjon: 'AI-generering feilet. Vennligst prøv igjen senere.'
87      };
88    }
89  }
```

Forklaring: Dette screenshotet viser den tekniske håndteringen av AI-svaret. Her bruker jeg regulære uttrykk (.replace()) for å fjerne DeepSeeks interne tankerekker (<think>-tags) og markdown-kodeblokker. Dette sikrer at applikasjonen ikke krasjer hvis AI-en gir et uventet format, og sørger for at bare den rene miljørapporten blir stående igjen for brukeren.

Benchmarking

Funksjonell beskrivelse:

Benchmarking-siden er utviklet som et klimadashbord, hvor formålet er å gi sanntidsinnsikt i bedriftens miljøutslipp. Funksjonaliteten er hovedsaklig delt inn i 3 hovedområder for å gi brukeren rask oversikt og mulighet til fordypning:

1. Status: Øverst på dashbordet vises bedriftens totale utslipp, gjennomsnittlig utslipp per prosjekt, antall prosjekte og antall leverandører.
2. Benchmarking: Løsningen sammenligner bedriftens gjennomsnittlige utslipp mot bransjenormalen, som både vises som tekstlig vurdering, og et smultringdiagram. Det er også vist frem oversikt over utslippene av alle prosjekter i et diagram.
3. Prosjektdetaljer: Det er også en tabell nederst som gir detaljert informasjon om hvert prosjekt, inkludert status, oppstartsdato og CO2 utslipp.

Visuell Beskrivelse:

Det har vært hovedfokus på å designe en brukervennlig og oversiktlig UX opplevelse.

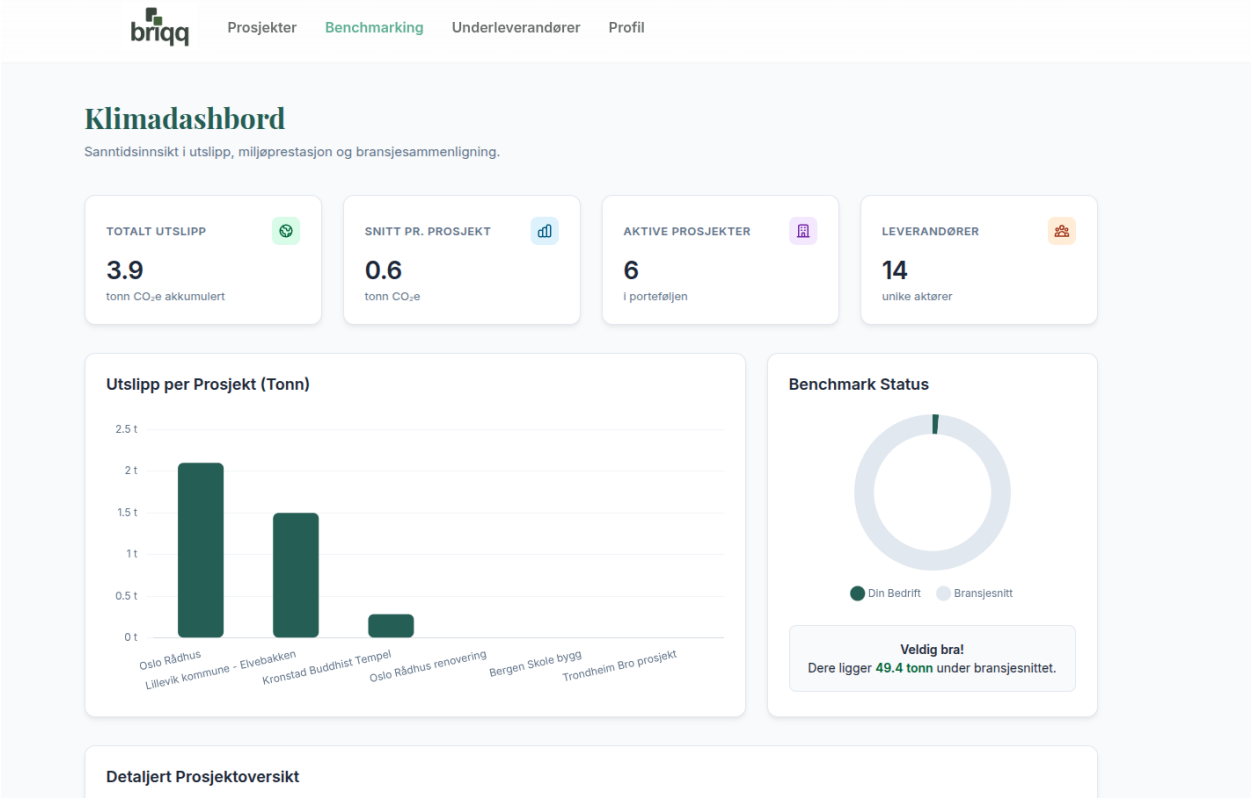
1. Fargebruk: Vi har brukt grønne farger for å signalisere miljø og soliditet, med hvit bakgrunn slik at det er enkelt å lese, og skape kontrast.
2. Layout: innhold er gruppert i et "card-based" design med fokus på tydelig gruppering av bokser, noe som gjør siden enkel å visuelt scanne for brukeren.
3. Status: Fargekoding på tabellen, der grønn er for aktiv, gul for planlagt og blå for ferdig, gjør det enkelt for brukeren å vite statusen på prosjektene.

Teknisk Beskrivelse:

Løsningen er en full-stack webapplikasjon som er bygget med javascript-teknologi på både client og serveren. Løsningen er delt inn i frontend som håndterer det visuelle, og backend som håndterer logikk, sikkerhet og data.

1. Frontend: Clientsiden er laget med ren javascript, HTML og CSS. Grafer er generert ved bruk av chart.js.
2. Backend: Serveren kjører på nodejs, med expressjs. Dette fungerer som applikasjonens API og håndterer kommunikasjon med databasen, auth av brukere og utsending av e-postinvitasjoner.

3. Dataflyt og API: Frontend henter data via fetch mot REST endepunkter. Serveren samler data fra ulike kilder, og returnerer den ferdige utregningen i benchmarking-dashboardet.



Sikkerhetsanalyse sikkerhet vurdering og implementasjon

Innledning

Denne rapporten redegjør for hvordan sikkerhet er ivaretatt i SmidigProsjekt-applikasjonen for Briqq, med særlig fokus på autentisering, håndtering av sensitive data og sikker bruk av kode. Formålet er å dokumentere hvilke sikkerhetstiltak som er implementert i løsningen, samt å vurdere hvordan disse tiltakene bidrar til å beskytte applikasjonen mot vanlige trusler. I tillegg identifiseres enkelte svakheter, som presenteres som anbefalinger for videre forbedring.

Applikasjonen er utviklet med Node.js og Express, og benytter en database for lagring av brukerinformasjon og prosjektrelaterte data. Vurderingene i rapporten tar utgangspunkt i anerkjente sikkerhetsprinsipper og anbefalinger fra OWASP Top 10.

Overordnet sikkerhetsdesign

Allerede i applikasjonens struktur er det gjort valg som legger til rette for god sikkerhet. Ansvarsområder er tydelig separert, der databasehåndtering, applikasjonslogikk og klientkode er delt opp i egne filer og kataloger. Dette bidrar til bedre kontroll over hvor sensitiv funksjonalitet befinner seg, og reduserer risikoen for at kritisk logikk eksponeres mot klienten. Ved å samle serverlogikk på backend og begrense frontend til presentasjon og brukerinteraksjon, følger applikasjonen prinsippet om at klienten aldri skal stoles på.

Autentisering og håndtering av passord

Et sentralt sikkerhetstiltak i applikasjonen er håndtering av brukernes passord. I stedet for å lagre passord i klartekst, benyttes hashing ved hjelp av bcrypt før passordene lagres i databasen. Dette innebærer at selv om databasen skulle bli kompromittert, vil angriperen ikke kunne lese eller bruke passordene direkte.

```
const bcrypt = require("bcrypt");

const hashedPassword = await bcrypt.hash(password, 10);
```

Applikasjonen benytter hashing av passord før lagring i databasen. Dette er et kritisk sikkerhetstiltak som sikrer at passord ikke lagres i klartekst.

Ved innlogging sammenlignes brukerens passord med hash i databasen:

```
const match = await bcrypt.compare(inputPassword, user.password);
```

Dette sikrer at passordet aldri sammenlignes i klartekst, og kun riktig bruker får tilgang

Denne implementasjonen er i tråd med anbefalinger fra OWASP og representerer et viktig tiltak for å ivareta konfidensialiteten til brukerkontoene.

Beskyttelse av sensitiv informasjon

Applikasjonen håndterer potensielt sensitiv informasjon, som brukerdata og prosjektrelaterte opplysninger. Det er derfor viktig at slik informasjon behandles på en forsvarlig måte. I løsningen er det lagt opp til at sensitiv informasjon ikke logges unødvendig, spesielt ikke i forbindelse med autentisering.

Fra app.js:

```
if (isProtected) {
        if (!req.cookies.EmailCookie) {
        console.log(` Uautorisert tilgangsforsøk til ${req.path}`);
        return res.redirect('/');
        }
}
```

Her logges kun et uautorisert tilgangsforsøk har skjedd, hvilken **URL/sti** som ble forsøkt åpnet.

Fra app.js:

```
return res.status(500).json({
        success: false,
        message: "Kunne ikke hente data"
});
```

- Brukeren får kun en generisk feilmelding
- Ingen tekniske detaljer, SQL-feil eller brukerdata eksponeres
- Eventuelle interne feil holdes på serversiden

Dette reduserer risikoen for informasjonsovereksponering, som er en kjent OWASP-sårbarhet.

Klient–server-sikkerhet

Frontend-koden i prosjektet inneholder kun presentasjonslogikk og enkel brukerinteraksjon. Ingen sensitive operasjoner eller sikkerhetskritiske avgjørelser utføres på klientsiden.

Fra script.js:

```
loginForm.addEventListener('submit', function(e) {

  e.preventDefault();

  const email = document.getElementById('email').value;

  const password = document.getElementById('password').value;



  if (email && password) {

        showToast('Innlogging vellykket!');

        setTimeout(()=> window.location.href = 'index.html', 700);

  }

});
```

Dette er kun UI-logikk (toast + redirect) og autentiserer ikke mot server. Det er bra at frontend ikke "bestemmer" tilgang til backend-data, men dette kan også bli en svakhet hvis det gir inntrykk av ekte innlogging. Reell autentisering må skje på server (API) og beskyttede ressurser må sjekke cookies/sesjon på backend.

Tilgangskontroll og autorisasjon

Løsningen kontrollerer tilgang til funksjonalitet basert på om brukeren er autentisert. Dette bidrar til å hindre at uautoriserte brukere får tilgang til beskyttede ressurser.

Vurdering av svakheter og forbedringspotensial

Selv om applikasjonen har en god grunnleggende sikkerhetsimplementasjon, finnes det enkelte områder som kan forbedres. Manglende rate limiting, sikkerhetsheadere og forbedret sesjonshåndtering er eksempler på dette.

Konklusjon

SmidigProsjekt viser en gjennomtenkt tilnærming til sikkerhet og følger flere etablerte beste praksiser for sikker webutvikling.

Beskrivelse av hvordan kjøre siden selv.

For å kjøre prosjektet lokalt på egen datamaskin må nyeste versjon av node være lastet ned. Github repoet må være lastet ned til en mappe på datamaskinen. Inne i prosjektmappen må man laste ned alle pakker tilhørende prosjektet med å kjøre "npm install". For at nettsiden skal funke må man ha en mongoDB bruker med tilgang til en cluster med navn "BriqqCluster" som har en database med navn "BriqqDatabase" denne databasen må inneholde tre collections med navn "ProjectsCollection", "SubmissionsCollection" og "UserCollection". Vi har valgt å bruke to forskjellige AI-modeller, en for miljørapporten og en for underleverandøranalysen. Du kan også velge å bruke en AI modell. Du trenger en API nøkkel som til AI modellen du velger å bruke. Du trenger også en dedikert e-post som kan sende ut skjemaer til underleverandørene. Når du har alt dette må du legge inn API nøkklene, eposten og passordet og mongoDB bruker URLen inn i "env" filen. Når alt dette er gjort så er nettsiden klar til å kjøres lokalt på datamaskinen.

2 Leveransetest

Formål og metode

Hovedformålet med leveransetesten var en avsluttende "selv-testing" for å verifisere at Briqq-løsningen oppfyller målene satt i Dokumentasjonen. Vi ønsket å bekrefte at vi ikke bare har levert ønsket funksjonalitet, men også nådd de tekniske målene om sikkerhet, feilhåndtering og en sømløs integrasjon mellom frontend og backend.

Gjennomføring

Vi utarbeidet et testprotokoll basert på sprintmålene for å kvalitetssikre hele systemet. Testen dekket tre hovedområder:

1. Integrasjon: Verifisering av at frontend og backend fungerer som en felles løsning.

2. Funksjonalitet: Gjennomgang av brukerreisen fra opprettelse av prosjekt til visning av benchmarking-data.

3. Teknisk kvalitet: Testing av sikkerhetsmekanismer (tilgangskontroll) og systemets håndtering av feil (feilhåndtering).

Resultat

Leveransetesten bekrefter at både funksjonelle og tekniske mål er nådd:

● Funksjonalitet: Benchmarking, prosjektstyring og oversikt over leverandører fungerer i henhold til kravene.

● Teknisk kvalitet: Backend og frontend er integrert på en god måte. Vi fikk verifisert at sikkerhetstiltakene stopper uautorisert tilgang, og feilhåndtering som kan resultere i crash.

● Prosess: Testen bekrefter også at målene om bedre struktur og dokumentasjon i Scrumwise har gitt resultater i form av en testbar og stabil løsning.

Vi konkluderer med at leveransen er godkjent og i mål.

Se vedlegg2 i WiseFlow for leveransetest.

3 Egen vurdering av resultat

Hva gruppen er fornøyd med

Samarbeid og teamdynamikk Gruppen fungerte godt sammen, både ved fysiske møter og online. Det var ingen konflikter mellom gruppemedlemmene, og alle bidro konstruktivt. Kommunikasjonen var effektiv, særlig når teamet møttes fysisk for å løse utfordringer sammen. Delegering av oppgaver og ansvar foregikk jevnt, og alle tok ansvar for sine arbeidsoppgaver.

Teknisk utvikling Den tekniske utviklingen av både frontend og backend gikk bra. Vi lyktes med å utvikle en fungerende plattform som integrerer AI-teknologi for miljørapportering. Bruken av GitHub ble bedre etter hvert sprint, noe som forbedret kodekvaliteten og samarbeidet rundt utviklingen.

Prosess og planlegging Vi kom i mål med det vi ønsket å oppnå i hver sprint. Oppgavene ble godt fordelt mellom medlemmene, og vi brukte Scrumwise effektivt til å holde oversikt. Den nyttige informasjonen vi samlet under research-fasen ga oss et solid grunnlag for utviklingen.

Kommunikasjon Vi kommuniserte godt som gruppe, delte mye av oppgavene, og alle deltok aktivt i diskusjoner både online og offline.

Hva som kunne vært bedre

Tidsplanlegging og estimering Vi estimerte for lite tid til flere oppgaver, noe som førte til tidspress underveis. Det var vanskelig å få alle tilpasset til de nye løsningene i tide, og mye tid gikk med til fletting av løsningene (merging) mot slutten av sprintene. En mer realistisk tidsbruk fra starten ville ha gitt mindre stress.

Tekniske utfordringer Kodingen kunne ha vært mer sammenhengende og strukturert. For dårlig bruk av GitHub tidlig i prosjektet gjorde det vanskeligere å samarbeide effektivt om koden. Skydatabase-tilgangen ble blokkert av brannmuren i MongoDB, noe som forsinket arbeidet. Finpussing av koden tok mer tid enn forventet.

Kommunikasjon og koordinering Kommunikasjonen mellom backend-utviklerne kunne vært bedre. Det var til tider litt vanskelig å få alle tilpasset til den nye løsningen, og ting endret seg plutselig, noe som krevde ekstra tilpasning.

Scrumwise-bruk Vi kunne ha fokusert mer på å skrive detaljerte oppgaver i Scrumwise fra begynnelsen av. Dette ville gitt bedre oversikt og struktur i arbeidet.

Forbedringsområder og læring

Planlegging Vi har lært viktigheten av realistisk tidsbruk og bedre målsetting for hver sprint. Å planlegge bedre tidsbruk og fordeling av oppgaver vil være essensielt i fremtidige prosjekter.

Verktøybruk GitHub må brukes mer systematisk fra starten for smidig utvikling. Bedre kodekvalitet og mindre finpussing på sene tidspunkt vil frigjøre tid til viktigere oppgaver.

Kommunikasjon Bedre kommunikasjon mellom backend-utviklerne og mer effektivt samarbeid mellom frontend og backend vil redusere problemer med integrasjon.

Scrumwise Å begynne med referatet tidligere og ha mer detaljerte oppgaver i Scrumwise fra dag én vil gi bedre oversikt og struktur.

Innsikt og forståelse

Gjennom prosjektet har vi erfart at god kommunikasjon og realistisk planlegging er avgjørende for suksess i et smidige utviklingsprosjekt. Vi har sett verdien av å møtes fysisk for å løse komplekse problemer raskt, samtidig som vi har lært at tekniske verktøy som GitHub må brukes konsekvent fra starten.

Vi har også innsett at AI-integrering i et byggeprosjekt krever grundig research og testing. Selv om vi møtte tekniske utfordringer som database-tilgang og kodeintegrasjon, lyktes vi med å levere en fungerende løsning. Erfaringene fra retrospektivene viser tydelig at kontinuerlig forbedring og åpen kommunikasjon er nøkkelen til bedre resultater i fremtidige sprinter.

4 Vedlegg

Kildekode

Se «Vedlegg1.zip» i WiseFlow – Kildekode

Leveransetest

Se «Vedlegg2.pdf» i WiseFlow - Leveransetest

Artefakter

Se «Vedlegg3.pdf» i WiseFlow – Skisser og prototyper

Se «Vedlegg4.pdf» i WiseFlow – Kontrakt

Referanser:

OWASP. (2025). OWASP Top 10 Web Application Security Risks. https://owasp.org/www-project-top-ten/

OWASP. (n.d). Application Security Verification Standard (ASVS). https://owasp.org/www-project-application-security-verification-standard/

MongoDB. (2024). MongoDB Security Best Practices. https://www.mongodb.com/docs/manual/security/

OpenAI. (2023). Prompt Injection and Safety Considerations. https://openai.com/index/prompt-injections/